

Rail Accident Damage Cost Prediction

Purpose

The purpose of this notebook is to evaluate a dataset of rail accidents and train regression models to predict the resulting damage cost (Total Damage Cost). The data is sourced from the Federal Railroad Administration (FRA) and the US Census Bureau. Each row contains the characteristics of the accident as described on the Rail Equipment Accident/Incident Report (FRA form 54), as well as the population and RUCC code of the area the accident occurred in. The goal is to understand what impact population has on the severity/cost of an accident, though this analysis mainly focuses on the accuracy of the models and the efficacy of using regression to predict accident costs.

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.io as pio
import plotly.graph_objects as go
```

Load Dataset

The processed dataset loaded here was created in the phase-1-proposal notebook. There, the datasets described above were loaded, cleaned, joined, and saved for analysis.

```
In [4]: # Set the default template globally
pio.templates.default = "plotly_white"

#Reads the dataframe from memory
df = pd.read_csv('../data/processed/rail_accident_pop.csv', low_memory = False)
#Display the first few rows
df.head()
```

Out[4]:

	Reporting Railroad Code	Reporting Railroad Name	Year	Accident Number	PDF Link	Accident Year	Acci Me
0	BNSF	BNSF Railway Company	2013	NW0813104	https://safetydata.fra.dot.gov/Officeofsafety/...	13	
1	CSX	CSX Transportation	2014	000131012	https://safetydata.fra.dot.gov/Officeofsafety/...	14	
2	CSX	CSX Transportation	2020	000191201	https://safetydata.fra.dot.gov/Officeofsafety/...	20	
3	CSX	CSX Transportation	2013	000121335	https://safetydata.fra.dot.gov/Officeofsafety/...	13	
4	SFRV	South Florida Regional Transportation Authority	2024	03172024	https://safetydata.fra.dot.gov/Officeofsafety/...	24	

5 rows × 164 columns

Some additional preprocessing was required to prepare the dataset for regression. Some numeric columns that were stored as strings and contained commas are converted to integers and floats. The hour the accident occurred was extracted from the accident time, as we expect the time of an accident may impact the resulting damage.

In [6]:

```
df['Date'] = pd.to_datetime(df.loc[:, 'Date'])
df['Time'] = pd.to_datetime(df.loc[:, 'Time'])

df['Hour'] = df['Time'].dt.hour
df['Persons Evacuated'] = df['Persons Evacuated'].str.replace(',', '').astype(int)
df['Gross Tonnage'] = df['Gross Tonnage'].str.replace(',', '').astype(int)
df['Equipment Damage Cost'] = df.loc[:, 'Equipment Damage Cost'].str.replace(',', '').astype(float)
df['Track Damage Cost'] = df.loc[:, 'Track Damage Cost'].str.replace(',', '').astype(float)
df['Total Damage Cost'] = df.loc[:, 'Total Damage Cost'].str.replace(',', '').astype(float)
```

C:\Users\Matt\AppData\Local\Temp\ipykernel_6640\313596752.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
df['Time'] = pd.to_datetime(df.loc[:, 'Time'])

Additionally, there are some rows of the dataset that describe duplicate accidents. We aggregated the damage cost columns across duplicated rows, then removed duplicates. This ensures we account for all damages that occurred in each accident.

```
In [8]: #Gets all accident numbers that appear more than once in the dataset
dup_accident_nums = df[df['Accident Number'].duplicated()][['Accident Number']]

#Selects all rows with duplicate accidents
dup_accidents = df.loc[df['Accident Number'].isin(dup_accident_nums), :].sort_values(by = 'Accid

#Selects all columns that describe a 'Cost'
cost_cols = [i for i in df.columns if 'Cost' in i]

#Aggregates the equipment costs and track damage costs among duplicate accidents
combined_cost_rows = dup_accidents.groupby(['Accident Number'])[ ['Equipment Damage Cost', 'Track

#Updates the main dataframe with the aggregated costs and drops duplicate accident rows
df_indexed = df.drop_duplicates(subset = 'Accident Number').set_index('Accident Number')
df_indexed.update(combined_cost_rows)

#Removes duplicate accidents from the main dataframe
df = df_indexed.reset_index()
```

Feature Analysis

Our initial EDA explored the relationships between RUCC code, damage cost, and accident cause type. In this analysis, we will focus on understanding the features that predict Total Damage Cost. The pivot table below shows the median damage cost by cause category and location class (urban/rural). It's interesting to note that Track accidents tend to have a higher cost, especially those that occur in Rural environments. Also, accidents tend to have lower costs overall in Urban environments.

```
In [10]: display(pd.pivot_table(df, index = 'Cause_Category', columns = 'Location_Class', values = 'Total
            margins = True, aggfunc = 'median').round(2))
```

	Location_Class	Rural	Urban	All
Cause_Category				
E		56401.0	40007.5	44470.0
H		36192.0	33464.0	34000.0
M		33019.0	28574.0	29565.0
S		37272.5	30000.0	31088.0
T		77275.0	41000.0	48839.0
All		44494.0	33846.5	35815.0

Here, I manually selected columns that could influence the target value. This list excludes any features unique to each accident, such as Accident Number or PDF Link. Any categorical columns are converted into factors, and any rows missing data are dropped from the dataframe.

```
In [12]: features = [
    'Reporting Railroad Code', 'Accident Month', 'Hour',
    'Accident Type Code', 'Hazmat Cars', 'Hazmat Cars Damaged', 'Hazmat Released Cars',
    'Temperature', 'Visibility Code', 'Weather Condition Code', 'Track Type Code', 'Trac
    'Equipment Type Code', 'Train Speed', 'Maximum Speed', 'Gross Tonnage', 'Signalizati
    'Head End Locomotives', 'Mid Train Manual Locomotives', 'Mid Train Remote Locomotive
    'Rear End Remote Locomotives', 'Derailed Head End Locomotives', 'Derailed Mid Train
    'Derailed Mid Train Remote Locomotives', 'Derailed Rear End Manual Locomotives', 'De
    'Derailed Mid Train Remote Locomotives', 'Derailed Rear End Manual Locomotives', 'De
```

```

        'Loaded Freight Cars', 'Loaded Passenger Cars', 'Empty Freight Cars', 'Empty Passeng
        'Derailed Loaded Freight Cars', 'Derailed Loaded Passenger Cars', 'Derailed Empty Fr
        'Derailed Empty Passenger Cars', 'Derailed Caboose', 'Accident Cause Code',
        'Engineers On Duty', 'Firemen On Duty', 'Conductors On Duty', 'Brakemen On Duty',
        'Railroad Employees Killed', 'Railroad Employees Injured', 'Passengers Killed', 'Pas
        'Others Killed', 'Others Injured', 'Total Persons Killed', 'Total Persons Injured',
        'Joint Track Class', 'Class Code', 'Class', 'FIPS', 'Population_2020', 'RUCC_2023',
    ]

targets = ['Total Damage Cost']

corr_df = df[features + targets]
corr_df = corr_df.dropna()

#Factorizes all non-numeric columns
corr_df.loc[:, list(corr_df.dtypes == 'object')] = corr_df.loc[:, list(corr_df.dtypes == 'object

#Tracks the integer values assigned to each categorical value of the categorical columns
factor_keys = df.loc[:, list(df.dtypes == 'object')].apply(lambda x: pd.factorize(x)[1])
cause_categories = factor_keys['Cause_Category']

corr_df.head()

#corr_df.to_csv('../data/processed/factorized_data.csv')

```

Out[12]:

	Reporting Railroad Code	Accident Month	Hour	Accident Type Code	Hazmat Cars	Hazmat Cars Damaged	Hazmat Released Cars	Persons Evacuated	State Code	Temperature	..
0	0	8	16	1.0	0	0	0	0	53	71	..
1	1	6	21	1.0	1	0	0	0	21	70	..
2	1	7	7	12.0	0	0	0	0	47	76	..
3	1	10	18	1.0	0	0	0	0	45	71	..
4	2	3	13	1.0	0	0	0	161	12	85	..

5 rows × 66 columns

Once all of the data is numeric, we calculated the correlations between each feature and the Total Damage cost for each of the five cause categories. The five best predictors for each category are listed below:

In [14]:

```

corr_features = {}

for i in range(5):
    print(cause_categories[i])
    corr_features[cause_categories[i]] = corr_df[corr_df['Cause_Category'] == i].corr()['Total D
    display(corr_features[cause_categories[i]][1:])
    print()

```

```

T
Derailed Loaded Freight Cars    0.550466
Passengers Killed                0.478532
Passengers Injured              0.457590
Maximum Speed                   0.432826
Railroad Employees Injured       0.430113
Name: Total Damage Cost, dtype: float64
M

```

```

Derailed Loaded Freight Cars    0.276297
Persons Evacuated               0.210195
Hazmat Cars Damaged            0.125951
Derailed Head End Locomotives   0.114566
Derailed Empty Freight Cars     0.113750
Name: Total Damage Cost, dtype: float64
H
Total Persons Killed            0.707704
Total Persons Injured           0.684548
Passengers Injured              0.612158
Passengers Killed               0.586299
Derailed Loaded Passenger Cars  0.585343
Name: Total Damage Cost, dtype: float64
E
Derailed Loaded Freight Cars    0.679232
Hazmat Released Cars            0.404360
Hazmat Cars Damaged             0.349096
Loaded Freight Cars             0.275814
Gross Tonnage                   0.260544
Name: Total Damage Cost, dtype: float64
S
Total Persons Injured           0.678720
Passengers Injured              0.639453
Derailed Loaded Passenger Cars  0.598711
Maximum Speed                   0.441847
Loaded Passenger Cars           0.373844
Name: Total Damage Cost, dtype: float64

```

The features that best predict the damage cost are those related to the number of injuries/deaths, the number of derailed cars, the speed of the train, and the tonnage the train is carrying. We'll aggregate those features and explore the relationship between each and the target using a scatterplot matrix.

```

In [16]: df['Total Empty Cars'] = df['Empty Freight Cars'] + df['Empty Passenger Cars']
df['Total Engine Cars'] = df[['Head End Locomotives', 'Mid Train Manual Locomotives', 'Mid Train
                             'Rear End Manual Locomotives', 'Rear End Remote Locomotives']].sum

df['Total Loaded Cars'] = df['Loaded Freight Cars'] + df['Loaded Passenger Cars']

df['Derailed Empty'] = df['Derailed Empty Freight Cars'] + df['Derailed Empty Passenger Cars']
df['Derailed Loaded'] = df['Derailed Loaded Freight Cars'] + df['Derailed Loaded Passenger Cars']
df['Derailed Locomotives'] = df[['Derailed Head End Locomotives', 'Derailed Mid Train Manual Loc
                                'Derailed Mid Train Remote Locomotives', 'Derailed Rear End Manual Locomotives'],

```

We also filtered out any outlier accidents, which were any accidents whose total damage cost was above the sum of the 75th percentile of costs plus 1.5 times the cost IQR.

```

In [18]: cost_iqr = df['Total Damage Cost'].quantile(0.75) - df['Total Damage Cost'].quantile(0.25)

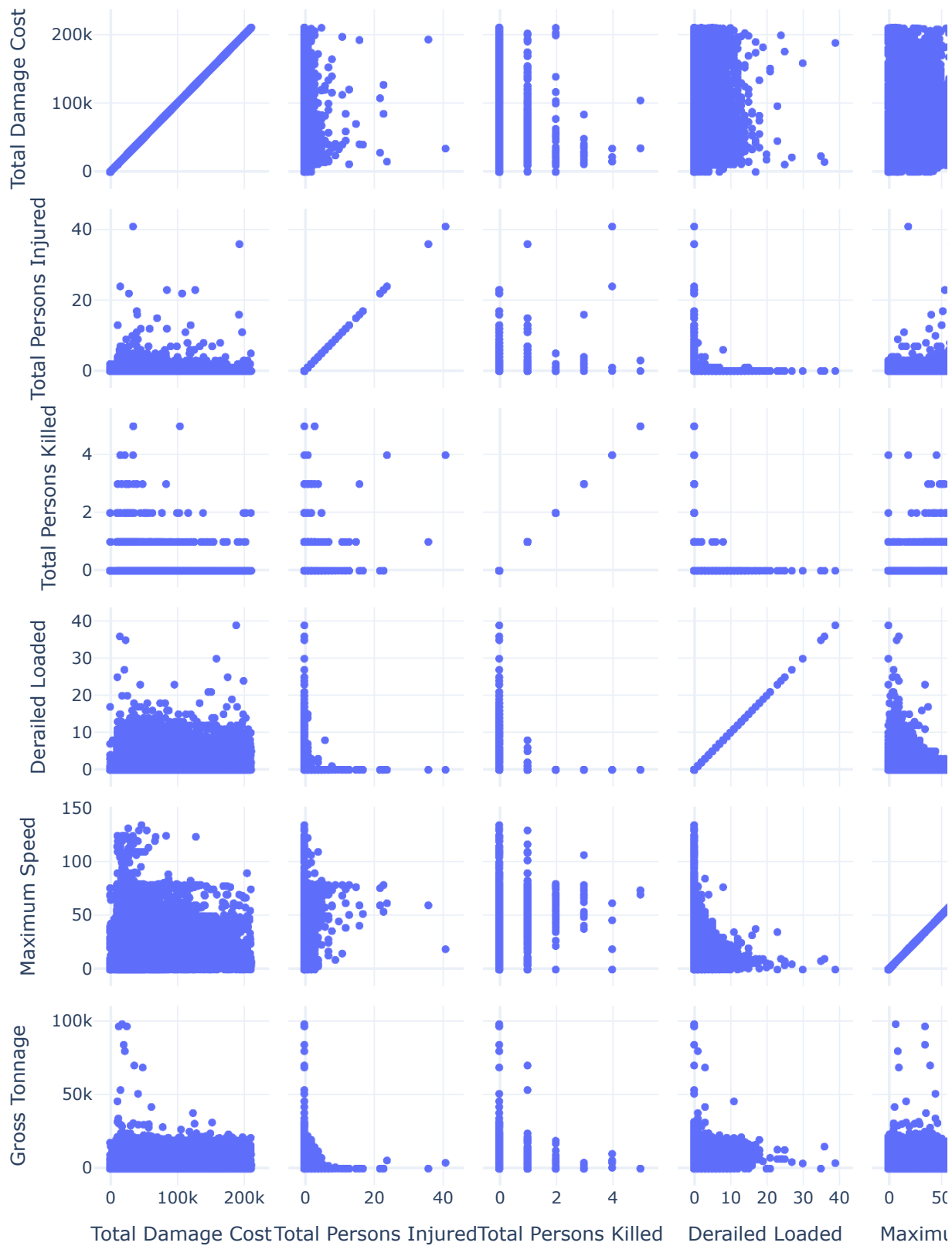
cost_upper_bound = df['Total Damage Cost'].quantile(0.75) + (1.5 * cost_iqr)
cost_lower_bound = df['Total Damage Cost'].quantile(0.25) - (1.5 * cost_iqr)

fig = px.scatter_matrix(df.loc[df['Total Damage Cost'] < cost_upper_bound,
                             ['Total Damage Cost', 'Total Persons Injured', 'Total Persons Killed', 'Derailed

fig.update_layout(
    height = 1000,
    width = 1000
)

fig.show()

```



Even after filtering out outliers, the right skew of the total damage costs makes it difficult to determine the relationship between our target and its predictors. We'll take the log of the total damage cost, then recreate this scatterplot matrix.

```
In [20]: fig = px.histogram(df.loc[df['Total Damage Cost'] < cost_upper_bound, 'Total Damage Cost'], titl
fig.update_layout(showlegend = False)
fig.show()
```



```
In [21]: df['Log Cost'] = np.log1p(df['Total Damage Cost'])

log_cost_iqr = df['Log Cost'].quantile(0.75) - df['Log Cost'].quantile(0.25)

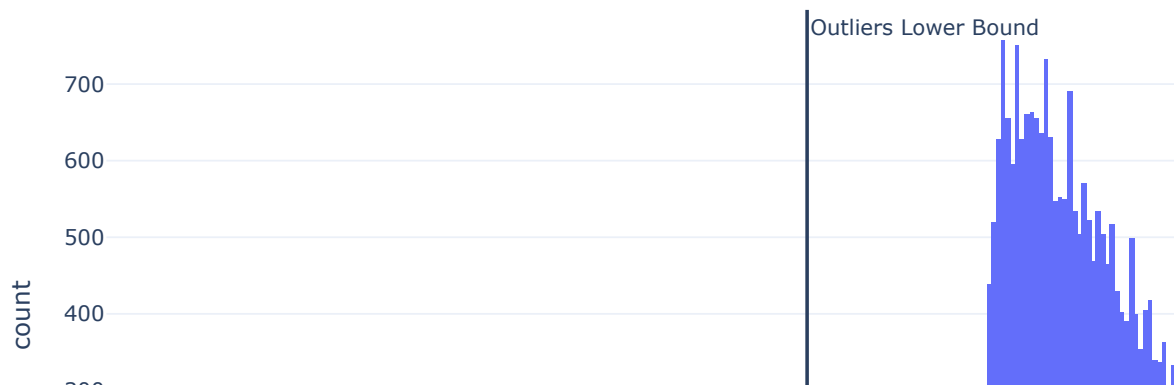
log_cost_upper_bound = df['Log Cost'].quantile(0.75) + (1.5 * log_cost_iqr)
log_cost_lower_bound = df['Log Cost'].quantile(0.25) - (1.5 * log_cost_iqr)

fig = px.histogram(df['Log Cost'], title = 'Log Cost Distribution')

fig.add_vline(x = log_cost_lower_bound, annotation_text="Outliers Lower Bound")
fig.add_vline(x = log_cost_upper_bound, annotation_text="Outliers Upper Bound")

fig.update_layout(showlegend = False)
fig.show()
```

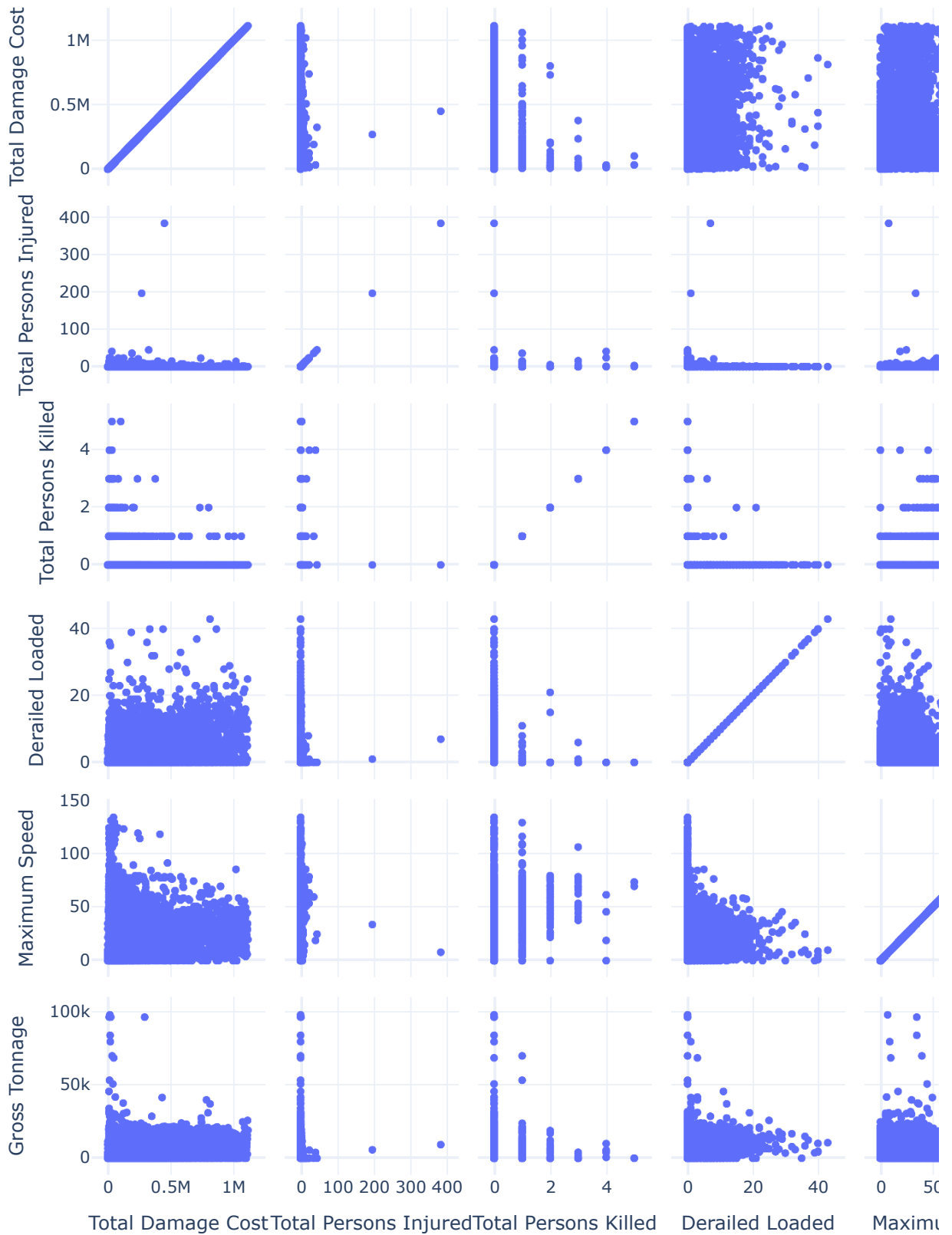
Log Cost Distribution



```
In [22]: fig = px.scatter_matrix(df.loc[(df['Log Cost'] >= log_cost_lower_bound) & (df['Log Cost'] <= log
                                     ['Total Damage Cost', 'Total Persons Injured', 'Total Persons Killed', 'Derailed

fig.update_layout(
    height = 1000,
    width = 1000
)

fig.show()
```

Using log costs in the scatterplot matrix shows little improvement. While correlations exist between our target and features, the regression will struggle due to a lack of linear relationships. We expect that nonlinear models (decision trees, random forest, etc) will perform better at predicting damage costs.

Model Training

Next, we will train and evaluate our machine learning models. To improve the performance and reduce the training time, we'll preprocess the data using the sklearn StandardScaler. 80% of our data will be used to train the models, and the remaining 20% will be the test set. To ensure our results are reproducible, we'll set a random seed of 521.

```
In [25]: from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

np.random.seed(521)

#Our feature subset includes the features that are correlated with the target, as well as additi
features_subset = [
    'Accident Month', 'Hour', 'Temperature', 'Visibility Code', 'Weather Condition Code',
    'Train Speed', 'Gross Tonnage',
    'Engineers On Duty', 'Firemen On Duty', 'Conductors On Duty', 'Brakemen On Duty',
    'Population_2020', 'RUCC_2023', 'Cause_Category',
    'Total Empty Cars', 'Total Engine Cars', 'Total Loaded Cars', 'Total Persons Killed'
]

targets = ['Log Cost']

#The only non-numeric column in this subset is Cause_Category, which we converted to numeric usin
x = pd.get_dummies(df[features_subset], columns = ['Cause_Category'], dtype = int).fillna(0)
x_cols = x.columns

#StandardScaler preprocessing (subtract the mean of each column and divide by the standard devia
scaler = StandardScaler()
x = pd.DataFrame(scaler.fit_transform(x))

x.columns = x_cols
y = df[targets]

#Splits the dataset into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2)

x_train.shape, y_train.shape
```

```
Out[25]: ((23893, 24), (23893, 1))
```

As we train our models, we'll use two plots to evaluate them. The prediction-error display plots the predicted values on the x axis and the actual values on the y-axis. A line that intersects the min and max actual value shows how well the predictions match the true damage costs. The code to create this visual in plotly is below in the prediction_error_display function.

Plot distributions displays overlapping histograms for the predicted and actual costs. We expect that the better a model can learn the distribution of costs, the better it will predict damage costs. We can also use these plots to predict whether or not a model is overfitting.

```
In [27]: from plotly.subplots import make_subplots

def prediction_error_display(y_test, y_pred):
    actual_predicted_df = pd.DataFrame({
        'Actual': y_test,
        'Predicted': y_pred
    })
```

```

fig = px.scatter(actual_predicted_df, 'Predicted', 'Actual', opacity = 0.7)

fig.add_trace(go.Scatter(
    x = [actual_predicted_df['Actual'].min(), actual_predicted_df['Actual'].max()],
    y = [actual_predicted_df['Actual'].min(), actual_predicted_df['Actual'].max()],
    mode = 'lines',
    line={ 'dash': 'dash' }
))

fig.update_layout(
    xaxis_title = 'Predicted Values', yaxis_title = 'Actual Values',
    height = 500, width = 500, showlegend = False
)

fig.add_annotation(
    x = actual_predicted_df['Predicted'].median(),           # X-coordinate in data space
    y = actual_predicted_df['Actual'].max() + 1,           # Y-coordinate in data space
    text="R2-Score: " + str(r2_score(y_test, y_pred)), # The text to display
    font=dict(size=12, color="black"),
    showarrow = False
)

return fig

def plot_distributions(x_train, y_train, x_test, y_test, reg):
    fig = make_subplots(rows = 2, cols = 1, subplot_titles = ['Training Cost Distribution', 'Test Cost Distribution'])

    fig.add_trace(go.Histogram(x = y_train, name = 'Actual', opacity = 0.5,
                               legendgroup='Actual', marker_color='blue'), row = 1, col = 1)
    fig.add_trace(go.Histogram(x = reg.predict(x_train).flatten(), name = 'Predicted', opacity = 0.5,
                               legendgroup='Predicted', marker_color='red'), row = 1, col = 1)

    fig.add_trace(go.Histogram(x = y_test, name = 'Actual', opacity = 0.5, showlegend = False,
                               legendgroup='Actual', marker_color='blue'), row = 2, col = 1)
    fig.add_trace(go.Histogram(x = reg.predict(x_test).flatten(), name = 'Predicted', opacity = 0.5,
                               legendgroup='Predicted', marker_color='red'), row = 2, col = 1)

    fig.update_layout(
        height = 500, width = 500,
        bargroup='overlay'
    )

    fig.update_xaxes(visible=False)
    fig.update_yaxes(visible=False)

    return fig

```

Most of our models are trained using sklearn. The below function fits an sklearn model to our training data, makes predictions using the testing data, then evaluates the root mean squared error, mean absolute error, and R2 score of those predictions. The function also displays the plots described above.

```

In [29]: from sklearn.metrics import root_mean_squared_error, mean_absolute_error, r2_score

def evaluate_model(model, x_train = x_train, y_train = y_train, x_test = x_test, y_test = y_test):
    x_train = x_train.to_numpy()
    y_train = y_train.to_numpy().flatten()

    x_test = x_test.to_numpy()
    y_test = y_test.to_numpy().flatten()

    reg = model.fit(x_train, y_train)

```

```
y_pred = reg.predict(x_test)

print('RMSE:', root_mean_squared_error(y_test, y_pred))
print('MAE:', mean_absolute_error(y_test, y_pred))
print('R2:', r2_score(y_test, y_pred))

fig = prediction_error_display(y_test, y_pred)
fig.show()

print()

fig = plot_distributions(x_train, y_train, x_test, y_test, reg)
fig.show()

return reg
```

Linear Regression

In [31]: `from sklearn.linear_model import LinearRegression`

```
lr_model = evaluate_model(LinearRegression())
```

RMSE: 1.2184326879000018

MAE: 0.9086457584799167

R2: 0.14883506383812894

